

CND Subsystem: July Task

Establishing the Telemetry and Data Foundation

July 6, 2026

Abstract

This document outlines the technical onboarding roadmap for the Data, Dashboard, and Computing (CND) subsystem. The primary objective is to establish baseline competencies in data pipeline architecture, visualization layers, and machine learning integration utilizing real-world atmospheric telemetry. Candidates are required to execute tasks of escalating complexity to demonstrate engineering proficiency.

1. EXECUTION DIRECTIVES

To ensure structural consistency across the subsystem, all candidates must adhere to the following operational directives:

- **Compulsory Baseline:** Task 1 is mandatory for all inductees to establish a standardized data pipeline.
- **Track Specialization:** Following the completion of Task 1, candidates must select either Track A (Task 2: Web Development) or Track B (Task 3: Machine Learning). Both can also be opted.
- **Progressive Complexity:** Tasks are tiered. Candidates are expected to advance from Level 1 (Base) toward Level 3 (Expert) within their chosen tracks to demonstrate advanced problem-solving capabilities. Only Level 1 is compulsory.

2. TARGET OPERATIONAL DATASET

Candidates will process data extracted from the Integrated Global Radiosonde Archive (IGRA), managed by NOAA.

2.1. Rationale

A radiosonde telemetry profile serves as an exact operational analogue to a CanSat mission. It sequentially records altitude, atmospheric pressure, ambient temperature, and relative humidity over a continuous vertical trajectory. Processing this uniform dataset ensures that all candidates build familiarity with structural flight data frameworks prior to handling physical payload deployments.

2.2. Assigned Station: Bhubaneswar

All candidates must use the radiosonde record for the **Bhubaneswar** station (WMO ID 42971) so that every submission is validated against the same ground truth.

- **Data portal:** <https://www.ncei.noaa.gov/pub/data/igra/data/data-por/>
- **Station (IGRA) ID:** INM00042971
- **File to download:** INM00042971-data.txt.zip
- **Direct link:** <https://www.ncei.noaa.gov/pub/data/igra/data/data-por/INM00042971-data.txt.zip>

How the code was identified: Every IGRA station has an 11-character ID: the first two letters are the FIPS country code (IN for India), the third character is a network code (M), and the final digits are the station's WMO number. Bhubaneswar's WMO upper-air station number is 42971, giving the ID INM00042971.

2.3. Understanding the Raw Data Format

The IGRA “data-por” files are plain-text, fixed-width files, **not** CSVs, so they cannot be loaded with a plain `pd.read_csv()` call without first specifying the column widths. The authoritative format specification is here: <https://www.ncei.noaa.gov/pub/data/igra/igra2-data-format.txt>. Each sounding (one balloon launch) consists of one **header record** followed by several **data (level) records**. Candidates must read this specification before writing their parser and should note the following:

- **Header record** (one line per launch, begins with #): contains the station ID, YEAR, MONTH, DAY, HOUR, release time (RELTIME), number of levels in the sounding (NUMLEV), and the launch LATITUDE/LONGITUDE.
- **Data records** (one line per altitude level within a sounding), with the columns most relevant to this task being:
 - PRESS – atmospheric pressure at that level, in Pa (divide by 100 for hPa/mb).
 - GPH – geopotential height (i.e. altitude) at that level, in meters.
 - TEMP – air temperature, in tenths of °C (divide by 10 for °C).
 - RH – relative humidity, in tenths of a percent (divide by 10 for %).
 - DPDP – dewpoint depression, in tenths of °C.
 - WDIR, WSPD – wind direction (degrees) and wind speed (tenths of m/s).
- Missing values are coded as -9999 or -8888; these must be handled as NaN, not as literal readings, during Task 1’s cleaning step.

Candidates are expected to write their own fixed-width parser (or use a documented column-spec, e.g. Pandas’ `read_fwf()`) based on this specification rather than guessing column positions. Next time onwards you have to search these specifications on your own.

3. TASK 1: PIPELINE ORCHESTRATION AND DATA SANITIZATION

Status: Compulsory. Focuses on data engineering fundamentals.

- **Level 1 (Foundational Ingestion):** Load the raw flight data utilizing the Python Pandas library. Implement interpolation or exclusion strategies to handle missing values. Extract and report the maximum altitude, total temperature variance, and the pressure delta between the ground and apogee. Generate a clean plot mapping Altitude against Temperature.
- **Level 2 (Functional Modularization):** Refactor the manual parsing script into a reusable, production-grade function (e.g., `clean_telemetry()`). The function must dynamically ingest any raw IGRA file and output a sanitized DataFrame without manual intervention.
- **Level 3 (Parallelization):** Scale the pipeline to process multiple historical telemetry logs simultaneously using parallel processing techniques. Serialize the final output into a highly optimized binary format (e.g., Parquet) to minimize data ingestion latency.

4. TASK 2: GROUND STATION UI FRAMEWORK

Status: Optional (Track A). Focuses on systems infrastructure and UI engineering.

- **Level 1 (Static Presentation):** Utilize the sanitized data from Task 1. Construct a single-page web application leveraging HTML, JavaScript, and Chart.js to render static visualizations of Altitude vs. Temperature and Altitude vs. Pressure.
- **Level 2 (Asynchronous Simulation):** Implement temporal rendering logic. Program the visualization elements to plot data nodes sequentially over a time loop, accurately simulating a live hardware ground-station telemetry stream.

- **Level 3 (Decoupled Architecture):** Architect a backend API utilizing FastAPI or Flask to serve the telemetry arrays. The frontend application must consume this stream asynchronously via JavaScript `fetch()` requests, mirroring a true mission control environment.

5. TASK 3: TELEMETRY INTELLIGENCE AND MODELING

Status: Optional (Track B). Focuses on statistical inference and predictive analytics.

- **Level 1 (Statistical Outlier Detection):** Implement an algorithmic checker to identify anomalies within the telemetry arrays. Calculate dynamic rolling means and flag any reading that deviates by more than two standard deviations (Z-score approach).
- **Level 2 (Predictive Modeling):** Develop a regression model mapping physical atmospheric equations. Train the algorithm to predict altitude vectors strictly from ambient barometric pressure records, and quantify the error rate against the true altitude data.
- **Level 3 (Unsupervised Classification):** Deploy an unsupervised machine learning architecture (e.g., K-Means or DBSCAN). Configure the model to dynamically discover atmospheric boundary vectors (segmenting the Troposphere from the Stratosphere) entirely based on temperature and pressure gradients, without supplying altitude labels.

6. SUBMISSION PROTOCOL

All source code and analytical logs must be committed to your personal GitHub repository, set to public so it can be reviewed. Include a README with your name and task details. If attempting Track B, also include the Google Colab notebook (.ipynb) for your ML work, in your repository itself